

Module 01 — Synthetic Data Generator

synthetic_data/generate_synthetic_data.py • 751 lines

In one sentence

Creates the entire dataset: 6,000 customers, 2,500 products, 138,345 interactions.

1. What this module does

This is the starting point of the whole project. The business case puts us at a new e-commerce platform that has **no real customer data yet**, so the data has to be invented before anything else can be built.

The important idea is that this file does **not** just produce random rows. Random data would be useless: every product would be equally popular, every customer equally active, and no model could learn anything. Instead the generator deliberately builds in the same awkward properties that real shopping data has - a few best-sellers dominating sales, most products barely selling, and some customers who have never bought anything.

Everything downstream depends on this file. If the data were not realistic, every model score reported later would be meaningless.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Synthetic data generation	Creating fake but realistic data with code	The whole module
Zipf / power law	A few things are hugely popular, most are not	Deciding how often each product is shown
Log-normal distribution	Most values small, a few very large	Product prices and customer activity levels
Sparsity	Almost every customer-product pair never happens	99.08% of the matrix is empty
Long tail	The large mass of rarely-bought products	80% of the catalogue
Cold start	New customers/products with no history	8% of users, 10% of items reserved
Latent factors	Hidden reasons behind a choice	Category preference and price sensitivity
Engagement funnel	Click leads to cart leads to purchase	How each interaction row is built
Gumbel top-k sampling	Fast way to pick weighted random items	Choosing which products each customer sees
Reproducibility (seeding)	Same random numbers every run	<code>np.random.default_rng(42)</code>

3. How it works, step by step

1. Set up paths and the seed. Paths are worked out from the file's own location, and one seeded random generator drives everything so the data is identical on every run.

2. Create 6,000 customers. Each gets an age, gender, city, a preferred category and - most importantly - a *segment* (Budget Shopper through to Luxury Enthusiast) which decides what price range

they shop in.

3. Create 2,500 products. Each gets a category, brand, price drawn from that category's own price band, a written description, and tags. The description text is what the content recommender later reads.

4. Decide how popular each product will be. A power law gives a few products huge exposure and most products very little. The weights are then shuffled so popularity has nothing to do with the product ID.

5. Reserve the cold-start slices. 10% of products are marked as launched in the last 30 days with almost no exposure; 8% of customers are held to 0-2 interactions.

6. Decide how active each customer is. Drawn from a log-normal, so there are a few power users and a long tail of very light users.

7. Pick which products each customer sees. Product popularity is tilted toward the customer's own category and price band, then Gumbel top-k picks the set in one fast vectorised step.

8. Build the funnel. One hidden 'affinity' score per pair drives dwell time, add-to-cart, purchase and star rating, so the four signals never contradict each other.

9. Add timestamps. Spread over 18 months but weighted toward recent months, and never earlier than the product's launch date.

10. Save three CSV files and print a validation summary.

4. Key functions

Function	What it does
generate_users()	Builds the 6,000-row customer table
generate_items()	Builds the 2,500-row product catalogue
draw_user_activity()	Decides how many interactions each customer gets
weighted_sample_without_replacement()	Fast weighted picking (Gumbel top-k)
generate_interactions()	Builds the 138,345-row interaction log
print_validation_summary()	Prints proof that the properties actually landed

5. Inputs, outputs and how to run

	Detail
Inputs	None - everything is generated
Outputs	data/raw/users.csv, items.csv, interactions.csv
Runtime	About 10 seconds
Run with	python synthetic_data/generate_synthetic_data.py

6. Key code

synthetic_data/generate_synthetic_data.py — weighted_sample_without_replacement()

```
474 | def weighted_sample_without_replacement(log_weights, k):  
475 |     """Gumbel top-k trick."""  
476 |     keys = log_weights + rng.gumbel(size=log_weights.shape[0])  
477 |     return np.argpartition(-keys, k - 1)[:k]
```

Adding random Gumbel noise to log-weights and taking the top k is the same as picking items one at a time by weight, but happens in a single fast step. The old loop-based version took minutes; this takes seconds.

7. Things to remember

Why the Zipf law has a +25 offset

Without the offset the most popular product would be expected to appear more times than there are customers. Since a customer can only interact with a product once, the top items would end up seen by *everybody* and would stop telling the models anything useful. Adding 25 flattens the very top while leaving the long tail unchanged.

Why one affinity score drives all four signals

If dwell time, cart, purchase and rating were each generated independently, you would get contradictory rows - somebody who left the page in two seconds but gave it five stars. Driving all four from one hidden score keeps them consistent, which is what lets a model learn a real pattern.

8. Likely questions

Q. Why generate data instead of downloading a real dataset?

A. The business case specifies a platform with no real interaction data. Generating it also means I know the ground truth - I built in exactly two hidden patterns (category preference and price sensitivity), so I can check whether each model actually recovers them.

Q. How do you know the data is realistic?

A. Ten checks in the EDA script, all passing: 99.08% sparsity, top 10% of products holding 62.9% of demand, Gini 0.716, cold-start cohorts present, and ratings skewed high with mean 3.91 - the J-shape real review data has.

Q. What is deliberately unrealistic?

A. The conversion rate. Real e-commerce converts about 2-3% of visits; this data converts 16%. At a realistic rate the log would contain only about 4,000 purchases, which is far too few to train or test on. It is a documented trade-off, not an oversight.